

STAR RX72N - rx_dmaca Library
1.0.0

Generated by Doxygen 1.16.1

1	Directory Hierarchy	1
1.1	Directories	1
2	File Index	3
2.1	File List	3
3	Directory Documentation	5
3.1	libs/rx_dmaca/inc Directory Reference	5
3.2	libs Directory Reference	5
3.3	libs/rx_dmaca Directory Reference	6
3.4	libs/rx_dmaca/src Directory Reference	6
4	File Documentation	7
4.1	libs/rx_dmaca/inc/rx_dmaca.h File Reference	7
4.1.1	Detailed Description	8
4.1.1.1	Architecture	8
4.1.1.2	DMA Start Sequence (mandatory order)	8
4.1.1.3	Thread Safety	8
4.1.1.4	NASA Power of 10	9
4.1.1.5	Driver Lifecycle State Machine	9
4.1.1.6	Transfer Sequence for rx_dmaca_transfer_poll()	10
4.1.2	Data Structure Documentation	11
4.1.2.1	struct rx_dmaca_config_t	11
4.1.3	Enumeration Type Documentation	12
4.1.3.1	rx_dmaca_defaults_t	12
4.1.4	Function Documentation	13
4.1.4.1	rx_dmaca_abort()	13
4.1.4.2	rx_dmaca_deinit()	14
4.1.4.3	rx_dmaca_init()	16
4.1.4.4	rx_dmaca_transfer_poll()	18
4.2	rx_dmaca.h	20
4.3	libs/rx_dmaca/src/rx_dmaca.c File Reference	24
4.3.1	Detailed Description	25
4.3.2	Enumeration Type Documentation	26
4.3.2.1	rx_dmaca_internal_constants_t	26
4.3.3	Function Documentation	26
4.3.3.1	internal_configure_channel()	26
4.3.3.2	internal_poll_act()	27
4.3.3.3	rx_dmaca_abort()	29
4.3.3.4	rx_dmaca_deinit()	30
4.3.3.5	rx_dmaca_init()	31
4.3.3.6	rx_dmaca_transfer_poll()	32
4.3.4	Variable Documentation	34
4.3.4.1	s_dmaca_initialized	34

4.3.4.2 <code>s_tag</code>	34
4.4 <code>rx_dmaca.c</code>	34

Chapter 1

Directory Hierarchy

1.1 Directories

inc	5
rx_dmaca.h	7
libs	5
rx_dmaca	6
inc	5
rx_dmaca.h	7
src	6
rx_dmaca.c	24
rx_dmaca	6
inc	5
rx_dmaca.h	7
src	6
rx_dmaca.c	24
src	6
rx_dmaca.c	24

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

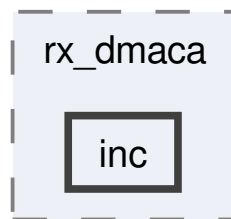
libs/rx_dmaca/inc/ rx_dmaca.h	
RX72N DMAC Polling-Mode Driver for CRC DMA Transfers	7
libs/rx_dmaca/src/ rx_dmaca.c	
RX72N DMAC Polling-Mode Driver Implementation	24

Chapter 3

Directory Documentation

3.1 libs/rx_dmaca/inc Directory Reference

Directory dependency graph for inc:

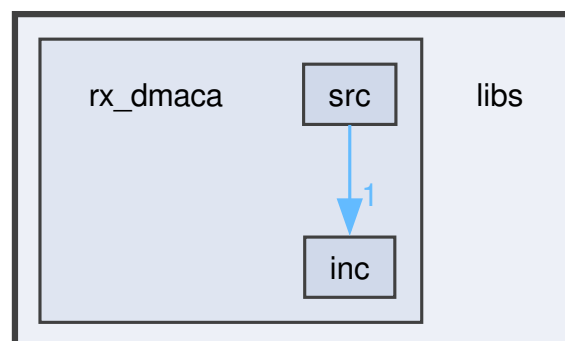


Files

- file [rx_dmaca.h](#)
RX72N DMAC Polling-Mode Driver for CRC DMA Transfers.

3.2 libs Directory Reference

Directory dependency graph for libs:

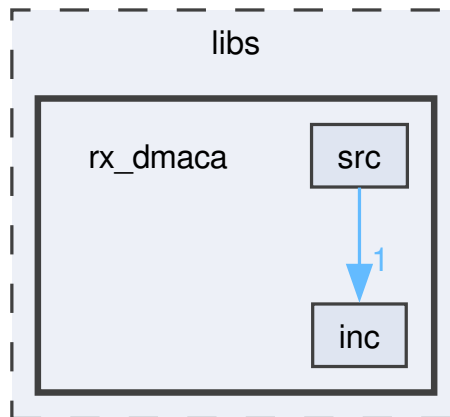


Directories

- directory [rx_dmaca](#)

3.3 libs/rx_dmaca Directory Reference

Directory dependency graph for rx_dmaca:

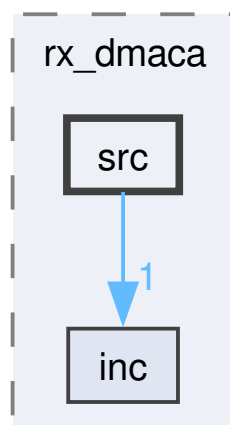


Directories

- directory [inc](#)
- directory [src](#)

3.4 libs/rx_dmaca/src Directory Reference

Directory dependency graph for src:



Files

- file [rx_dmaca.c](#)
RX72N DMAC Polling-Mode Driver Implementation.

Chapter 4

File Documentation

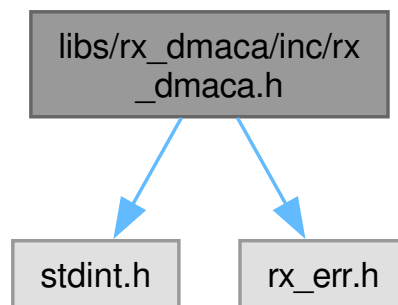
4.1 libs/rx_dmaca/inc/rx_dmaca.h File Reference

RX72N DMAC Polling-Mode Driver for CRC DMA Transfers.

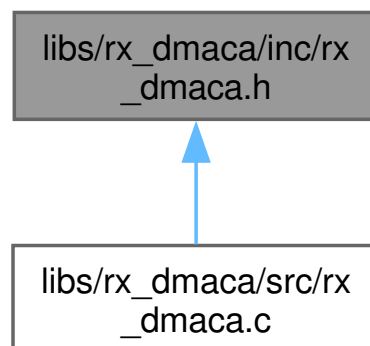
```
#include <stdint.h>
```

```
#include "rx_err.h"
```

Include dependency graph for rx_dmaca.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [rx_dmaca_config_t](#)
Configuration for a single DMAC polling transfer.

Enumerations

- enum [rx_dmaca_defaults_t](#): uint32_t { [k_dma_crc_timeout_cycles](#) = 50000U, [k_dma_channel_crc](#) = 0U }

Default constants for the DMAC-CRC integration.

Functions

- [rx_err_t rx_dmaca_init](#) (void)
Initialize the DMAC peripheral (enable module clock, set DMST=1).
- [rx_err_t rx_dmaca_transfer_poll](#) (const [rx_dmaca_config_t](#) *config)
Transfer data from src to a fixed destination via DMA (polling).
- [rx_err_t rx_dmaca_abort](#) (uint8_t channel)
Abort an in-progress DMA transfer on a channel.
- [rx_err_t rx_dmaca_deinit](#) (void)
Deinitialize DMAC (clear DMST, lock MSTPCRA).

4.1.1 Detailed Description

RX72N DMAC Polling-Mode Driver for CRC DMA Transfers.

Provides a minimal polling-mode DMA driver for the RX72N DMAC peripheral. Designed for use by [rx_crc](#) when offloading data-to-CRC DIR transfers via DMA, freeing the CPU from byte-at-a-time writes.

4.1.1.1 Architecture

```
rx_dmaca_transfer_poll()
|
+--> Configure DMSAR / DMDAR / DMCRA / DMTMD / DMAMD
+--> Set DMCNT.DTE = 1
+--> Set DMREQ.SWREQ = 1 (software trigger)
+--> Poll DMSTS.ACT until 0 or timeout
+--> Clear DMCNT.DTE
```

4.1.1.2 DMA Start Sequence (mandatory order)

1. Set DMAST.DMST = 1 (module-level start) – done once in [rx_dmaca_init\(\)](#)
2. Set DMCNT.DTE = 1 (channel enable)
3. Set DMREQ.SWREQ = 1 (trigger transfer)

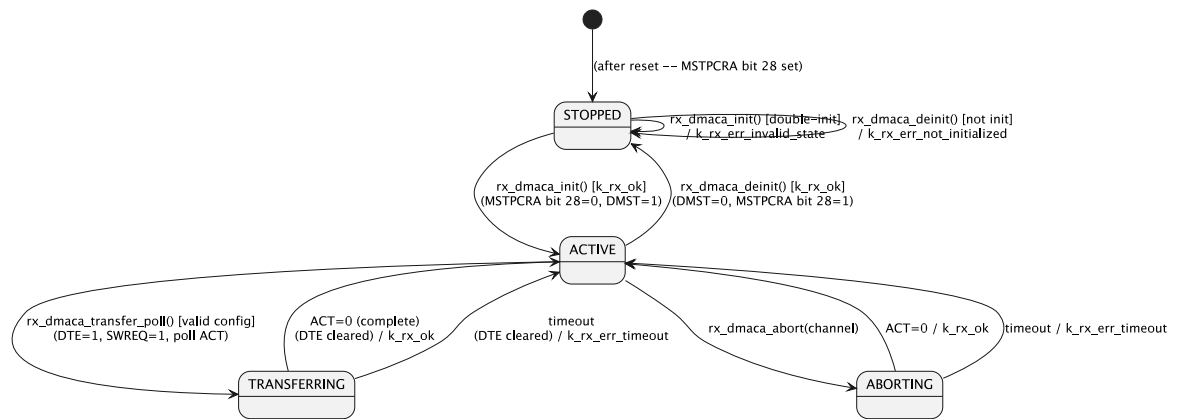
4.1.1.3 Thread Safety

[rx_dmaca_transfer_poll\(\)](#) is NOT thread-safe. The caller must ensure that only one transfer is active per channel at a time.

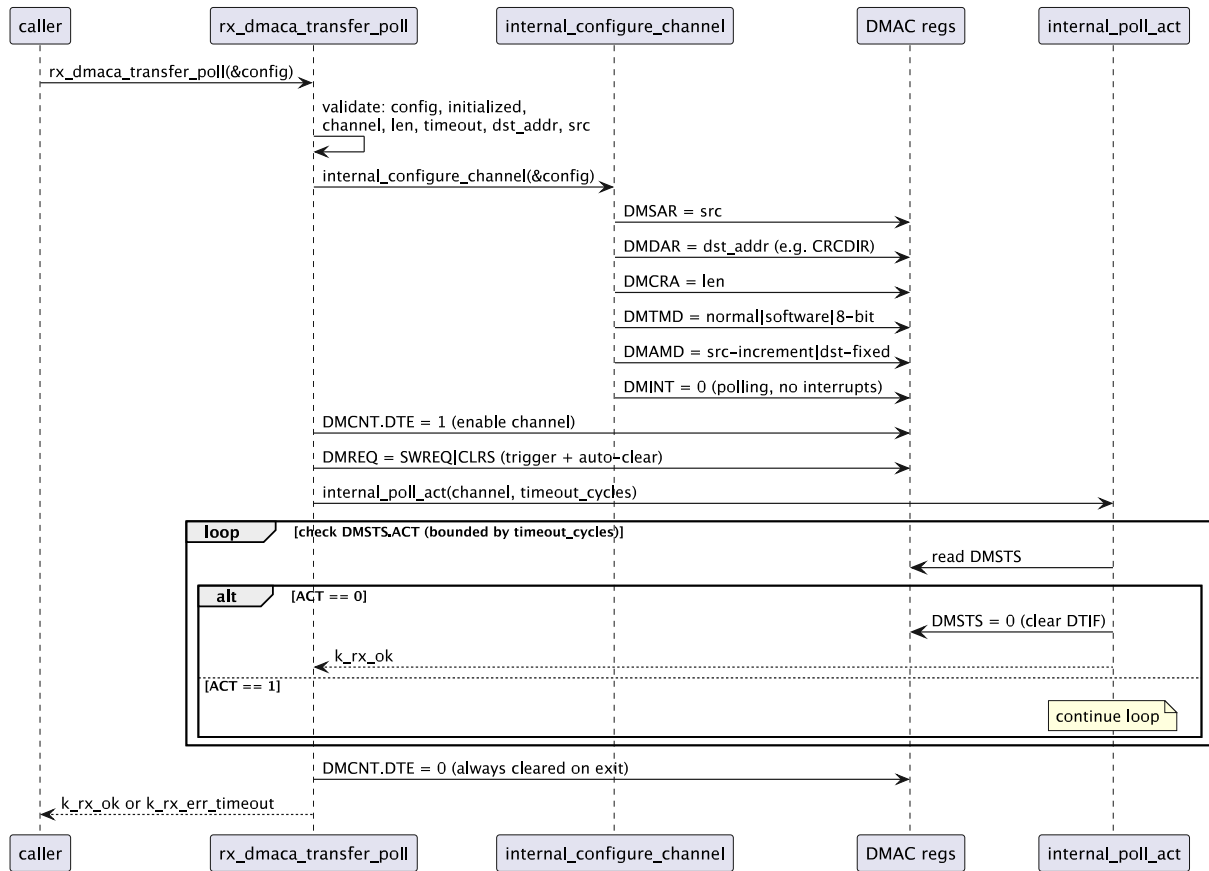
4.1.1.4 NASA Power of 10

Rule	Status	Notes
1. Simple flow	PASS	No goto/recursion
2. Bounded loops	PASS	Timeout loop bounded by timeout__cycles
3. No dynamic mem	PASS	Zero malloc
4. Short functions	PASS	All <= 60 lines
5. Assertions	PASS	NULL checks + range checks
6. Small scope	PASS	Vars declared at first use
7. Check returns	PASS	All returns checked
8. Limit preprocessor	PASS	C23 typed enums
9. Restrict pointers	WARN	Function pointers via bus interface pattern
10. Compiler warnings	PASS	-Wall -Wextra -Werror

4.1.1.5 Driver Lifecycle State Machine



4.1.1.6 Transfer Sequence for rx_dmaca_transfer_poll()



See also

`rx_crc.h` CRC library that uses this driver for `hw_dma` backend

`rx72n_dmac_regs.h` Hardware register definitions

Author

Locked, Inc.

Date

2026-03-05

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Definition in file [rx_dmaca.h](#).

4.1.2 Data Structure Documentation

4.1.2.1 struct rx_dmaca_config_t

Configuration for a single DMAC polling transfer.

Describes one DMA transfer from a source buffer to a fixed destination address (e.g., CRCDIR at 0x00088284). All fields must be set before calling [rx_dmaca_transfer_poll\(\)](#).

Invariant

channel must be in [0, 7]

src must not be NULL

len must be in [1, 65535]

timeout_cycles must be > 0

```
rx_dmaca_config_t cfg = {
    .channel    = k_dma_channel_crc,
    .src        = data,
    .len        = len,
    .dst_addr   = k_crcdir_addr,
    .timeout_cycles = k_dma_crc_timeout_cycles,
};
rx_err_t err = rx_dmaca_transfer_poll(&cfg);
```

See also

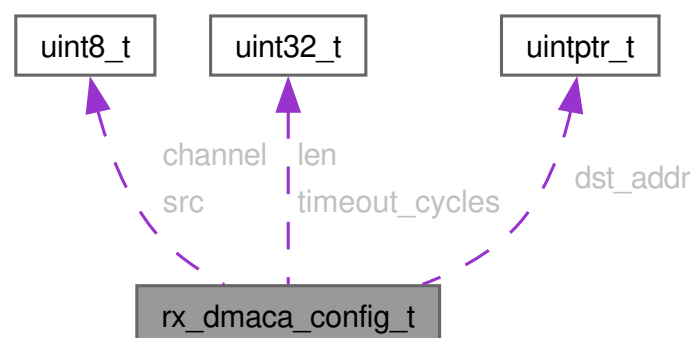
[rx_dmaca_transfer_poll\(\)](#) Uses this config

Since

Version 1.0.0

Definition at line 187 of file [rx_dmaca.h](#).

Collaboration diagram for rx_dmaca_config_t:



Data Fields

<code>uint8_t</code>	<code>channel</code>	DMA channel (0-7); 0 = highest priority
<code>uintptr_t</code>	<code>dst_addr</code>	Destination address (fixed, e.g., CRCDIR)

uint32_t	len	Number of bytes to transfer [1, 65535]
const uint8_t *	src	Source buffer pointer (incremented by DMA)
uint32_t	timeout_cycles	Max poll iterations before timeout error

4.1.3 Enumeration Type Documentation

4.1.3.1 rx_dmaca_defaults_t

enum [rx_dmaca_defaults_t](#) : uint32_t

Default constants for the DMAC-CRC integration.

Provides compile-time constants for CRC-specific DMA usage.

Since

Version 1.0.0

Enumerator

k_dma_crc_timeout_cycles	Timeout cycles for DMA polling (208 us at 240 MHz, 10x margin). Based on worst-case DMA transfer of 65535 bytes at 8-bit mode. At ~240 MHz with ~6 cycles/byte, 65535 bytes ~ 1.6 ms = 384000 cycles. 50000 cycles gives approximately 208 us which covers typical CRC buffers. Value: 50000
k_dma_channel_crc	DMAC channel reserved for CRC transfers. Channel 0 has highest priority among the 8 DMAC channels. Value: 0

Definition at line 134 of file [rx_dmaca.h](#).

```

00134         : uint32_t {
00135     /**
00136      * @brief Timeout cycles for DMA polling (208 us at 240 MHz, 10x margin)
00137      * @details
00138      * Based on worst-case DMA transfer of 65535 bytes at 8-bit mode.
00139      * At ~240 MHz with ~6 cycles/byte, 65535 bytes ~ 1.6 ms = 384000 cycles.
00140      * 50000 cycles gives approximately 208 us which covers typical CRC buffers.
00141      * @par Value: 50000
00142      */
00143     k_dma_crc_timeout_cycles = 50000U,
00144
00145     /**
00146      * @brief DMAC channel reserved for CRC transfers
00147      * @details Channel 0 has highest priority among the 8 DMAC channels.
00148      * @par Value: 0
00149      */
00150     k_dma_channel_crc = 0U,
00151 } rx_dmaca_defaults_t;
```


4.1.4 Function Documentation

4.1.4.1 rx'dmaca'abort()

```
rx_err_t rx_dmaca_abort (
    uint8_t channel) [nodiscard]
```

Abort an in-progress DMA transfer on a channel.

Clears DMCNT.DTE immediately, then polls until DMSTS.ACT = 0 to ensure the transfer has stopped before returning.

Parameters

in	channel	Channel to abort (0-7)
----	---------	------------------------

Returns

rx_err_t

Return values

k_rx_ok	Channel stopped
k_rx_err_invalid_arg	channel >= 8
k_rx_err_not_initialized	rx_dmaca_init() not called
k_rx_err_timeout	ACT bit did not clear within timeout

Precondition

[rx_dmaca_init\(\)](#) must have been called
channel must be in [0, 7]

Postcondition

DMCNT.DTE = 0 for the specified channel
DMSTS.ACT = 0 (transfer stopped)

Since

Version 1.0.0

Abort an in-progress DMA transfer on a channel.

Immediately clears DTE to disable the channel, then polls the ACT bit until the active transfer drains or the internal abort timeout expires. Returns k_rx_ok when the channel is confirmed idle.

Precondition

`rx_dmaca_init()` must have been called successfully
channel must be in `[0, k_dmac_channel_count - 1]`

Postcondition

DTE cleared for the specified channel
Channel ACT bit is 0 on `k_rx_ok` return

Note

Not thread-safe; do not call concurrently on the same channel

Since

Version 1.0.0

Definition at line 330 of file `rx_dmaca.c`.

```
00331 {
00332  /* Pre-conditions: validate inputs (NASA Rule 5) */
00333  if (!s_dmaca_initialized) {
00334    rx_log_error(s_tag, "DMAC not initialized");
00335    return k_rx_err_not_initialized;
00336  }
00337
00338  if (channel >= k_dmac_channel_count) {
00339    return k_rx_err_invalid_arg;
00340  }
00341
00342  /* Clear DTE to disable channel immediately */
00343  dmac_channel(channel)->dmcnt = 0U;
00344
00345  /* Wait for active transfer to drain (polls ACT bit) */
00346  return internal_poll_act(channel, k_dmaca_abort_timeout_cycles);
00347 }
```

References `internal_poll_act()`, `k_dmaca_abort_timeout_cycles`, `s_dmaca_initialized`, and `s_tag`.

Here is the call graph for this function:

4.1.4.2 `rx_dmaca_deinit()`

```
rx_err_t rx_dmaca_deinit (
    void ) [nodiscard]
```

Deinitialize DMAC (clear DMST, lock MSTPCRA).

Sets `DMAST.DMST = 0` then re-enables module stop (`MSTPCRA` bit 28 = 1) to gate the DMAC clock for power saving.

Returns

`rx_err_t`

Return values

<code>k_rx_ok</code>	Peripheral deinitialized
<code>k_rx_err_not_initialized</code>	Not initialized

Precondition

`rx_dmaca_init()` must have been called
No transfers are in progress on any channel

Postcondition

DMAST.DMST = 0 (all channels suspended)
MSTPCRA bit 28 = 1 (module stopped)

Warning

Calling `deinit` while a DMA transfer is in progress results in undefined behavior and potential data corruption at the destination address.

Since

Version 1.0.0

Deinitialize DMAC (clear DMST, lock MSTPCRA).

Clears DMAST.DMST (suspends all channels), unlocks PRCR, sets MSTPCRA bit 28 to gate the DMAC module clock, re-locks PRCR, and marks the driver as uninitialized. Returns `k_rx_err_not_initialized` if called without a prior `rx_dmaca_init()`.

Precondition

`rx_dmaca_init()` must have been called successfully
No DMA transfers should be in progress (all channels idle)

Postcondition

`s_dmaca_initialized` set to false
DMAC module clock disabled (MSTPCRA bit 28 set)

Note

Not thread-safe; call during single-threaded system teardown

Since

Version 1.0.0

Definition at line 367 of file [rx_dmaca.c](#).

```

00368 {
00369     /* Pre-condition: must be initialized (NASA Rule 5) */
00370     if (!s_dmaca_initialized) {
00371         rx_log_error(s_tag, "DMAC not initialized");
00372         return k_rx_err_not_initialized;
00373     }
00374
00375     /* Clear DMST to suspend all channels */
00376     *dmac_dmast_reg() = 0U;
00377
00378     /* Re-enable module stop to gate DMAC clock (power saving) */
00379     *prcr_reg() = k_rx_prcr_unlock_prc1;
00380     system_regs()->mstpcra |= k_dmac_mstpcra_bit;
00381     *prcr_reg() = k_rx_prcr_lock;
00382
00383     /* Post-condition: module stopped */
00384     s_dmaca_initialized = false;
00385     return k_rx_ok;
00386 }

```

References [s_dmaca_initialized](#), and [s_tag](#).

4.1.4.3 rx_dmaca_init()

```

rx_err_t rx_dmaca_init (
    void ) [nodiscard]

```

Initialize the DMAC peripheral (enable module clock, set DMST=1).

Enables the DMAC module by clearing MSTPCRA bit 28 (module stop) then setting DMAST.DMST = 1 to allow channel operation.

Re-initialization returns `k_rx_err_invalid_state` (not `k_rx_ok`) to alert callers of unexpected double-init. Callers that need idempotent behavior should check the return value and treat `k_rx_err_invalid_state` as acceptable.

Returns

`rx_err_t`

Return values

<code>k_rx_ok</code>	Peripheral initialized and ready
<code>k_rx_err_invalid_state</code>	Already initialized

Precondition

System clock (PCLKA) must be running

Protection registers must be accessible

Postcondition

MSTPCRA bit 28 cleared (DMAC clock enabled)

DMAST.DMST = 1 (module operating)

Note

Not thread-safe; call during single-threaded system init

Warning

Do not call from interrupt context

Since

Version 1.0.0

Initialize the DMAC peripheral (enable module clock, set DMST=1).

Unlocks PRCR, enables the DMAC module clock (clears MSTPCRA bit 28), starts the DMAC module (sets DMAST.DMST = 1), and marks the driver as initialized. Returns `k_rx_err_invalid_state` on double-init without touching hardware.

Precondition

DMAC must not already be initialized (`s_dmaca_initialized == false`)

System clock (PCLKA) must be configured and running

Postcondition

`s_dmaca_initialized` set to true

DMAC module clock enabled and DMAST.DMST = 1

Note

Not thread-safe; call during single-threaded system initialization

Since

Version 1.0.0

Definition at line 186 of file [rx_dmaca.c](#).

```
00187 {
00188     /* Pre-condition: detect double-init (NASA Rule 5) */
00189     if (s_dmaca_initialized) {
00190         return k_rx_err_invalid_state;
00191     }
00192
00193     /* Enable DMAC module clock (clear MSTPCRA bit 28) */
00194     *prcr_reg() = k_rx_prcr_unlock_prc1;
00195     system_regs()->mstpcra &= ~k_dmac_mstpcra_bit;
00196     *prcr_reg() = k_rx_prcr_lock;
00197
00198     /* Start DMAC module (DMAST.DMST = 1) */
00199     *dmac_dmast_reg() = k_dmast_dmst;
00200
00201     /* Post-condition: module operational */
00202     s_dmaca_initialized = true;
00203     return k_rx_ok;
00204 }
```

References [s_dmaca_initialized](#).

4.1.4.4 rx_dmaca_transfer_poll()

```
rx_err_t rx_dmaca_transfer_poll (
    const rx_dmaca_config_t * config) [nodiscard]
```

Transfer data from src to a fixed destination via DMA (polling).

Configures DMAC channel, triggers a software-initiated transfer, then polls DMSTS.ACT until transfer completes or timeout expires. Clears DTE on completion or error.

Transfer size (DMTMD.SZ) is always 8-bit (k_dmtmd_sz_8bit) for CRCDIR compatibility. The CRCA peripheral processes 8-bit bytes regardless of the 32-bit CRC polynomial selected.

Parameters

in	config	Transfer configuration (channel, src, len, dst, timeout)
----	--------	--

Returns

rx_err_t

Return values

k_rx_ok	Transfer complete
k_rx_err_null_ptr	config is NULL
k_rx_err_invalid_arg	channel >= 8, len == 0, len > 65535, timeout_cycles == 0 or > k_dmaca_timeout_cycles_max, or dst_addr == 0
k_rx_err_not_initialized	rx_dmaca_init() not called
k_rx_err_timeout	Transfer did not complete within timeout_cycles

Precondition

rx_dmaca_init() must have been called
 config->src must point to valid readable memory of config->len bytes

Postcondition

On success, all config->len bytes written to config->dst_addr
 DMCNT.DTE cleared regardless of success or failure

Note

Not thread-safe on the same channel

Since

Version 1.0.0

Transfer data from src to a fixed destination via DMA (polling).

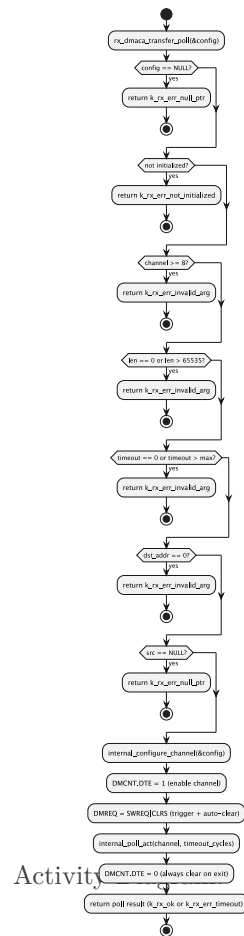
Validates all inputs, configures the DMA channel registers via [internal_configure_channel\(\)](#), starts the transfer (DTE=1, SWREQ=1), and polls the ACT bit until completion or timeout. Clears DTE on exit regardless of outcome.

Precondition

`rx_dmaca_init()` must have been called successfully
`config->src` must point to at least `config->len` bytes of readable memory

Postcondition

DMA channel DTE cleared (channel idle) regardless of return value
 On `k_rx_ok`, destination memory (`config->dst_addr`) contains transferred data



Note

Not thread-safe; concurrent calls on the same channel corrupt the transfer

Since

Version 1.0.0

Definition at line 266 of file `rx_dmaca.c`.

```

00267 {
00268     /* Pre-conditions: validate all inputs (NASA Rule 5) */
00269     RX_CHECK_NULL_PTR(config, s_tag, "config pointer is NULL");
00270
00271     if (!s_dmaca_initialized) {
00272         return k_rx_err_not_initialized;

```

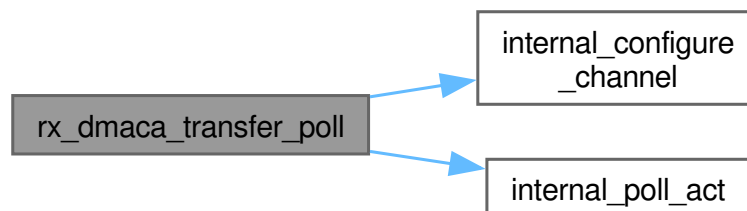
```

00273 }
00274
00275 if (config->channel >= k_dmac_channel_count) {
00276     return k_rx_err_invalid_arg;
00277 }
00278
00279 if (config->len == 0U || config->len > k_dmaca_len_max) {
00280     return k_rx_err_invalid_arg;
00281 }
00282
00283 if (config->timeout_cycles == 0U || config->timeout_cycles > k_dmaca_timeout_cycles_max) {
00284     return k_rx_err_invalid_arg;
00285 }
00286
00287 if (config->dst_addr == 0U) {
00288     return k_rx_err_invalid_arg;
00289 }
00290
00291 RX_CHECK_NULL_PTR(config->src, s_tag, "src pointer is NULL");
00292
00293 /* Configure channel registers */
00294 internal_configure_channel(config);
00295
00296 /* DMA start sequence (mandatory order per HW manual section 18): */
00297 /* 1. DTE = 1 (channel enable) */
00298 dmac_channel(config->channel)->dmcnt = k_dmcnt_dte;
00299
00300 /* 2. SWREQ = 1 + CLRS = 1 (trigger and auto-clear) */
00301 dmac_channel(config->channel)->dmreq = (uint8_t)(k_dmreq_swreq | k_dmreq_clrs);
00302
00303 /* Poll until complete or timeout */
00304 rx_err_t ret = internal_poll_act(config->channel, config->timeout_cycles);
00305
00306 /* Post-condition: clear DTE regardless of outcome (NASA Rule 5) */
00307 dmac_channel(config->channel)->dmcnt = 0U;
00308
00309 return ret;
00310 }

```

References [rx_dmaca_config_t::channel](#), [rx_dmaca_config_t::dst_addr](#), [internal_configure_channel\(\)](#), [internal_poll_act\(\)](#), [k_dmaca_len_max](#), [k_dmaca_timeout_cycles_max](#), [rx_dmaca_config_t::len](#), [s_dmaca_initialized](#), [s_tag](#), [rx_dmaca_config_t::src](#), and [rx_dmaca_config_t::timeout_cycles](#).

Here is the call graph for this function:



4.2 rx_dmaca.h

[Go to the documentation of this file.](#)

```

00001 /**
00002  * @file rx_dmaca.h
00003  * @brief RX72N DMAC Polling-Mode Driver for CRC DMA Transfers
00004  *
00005  * @details
00006  * Provides a minimal polling-mode DMA driver for the RX72N DMAC peripheral.
00007  * Designed for use by rx_crc when offloading data-to-CRCDIR transfers via DMA,
00008  * freeing the CPU from byte-at-a-time writes.
00009  *
00010  * ## Architecture
00011  *
00012  * ```
00013  * rx_dmaca_transfer_poll()
00014  * |
00015  * +--> Configure DMSAR / DMDAR / DMCRA / DMTMD / DMAMD
00016  * +--> Set DMCNT.DTE = 1

```



```

00017 *   +--> Set DMREQ.SWREQ = 1 (software trigger)
00018 *   +--> Poll DMSTS.ACT until 0 or timeout
00019 *   +--> Clear DMCNT.DTE
00020 *   ...
00021 *
00022 * ## DMA Start Sequence (mandatory order)
00023 *
00024 * 1. Set `DMAST.DMST = 1` (module-level start) -- done once in rx_dmaca_init()
00025 * 2. Set `DMCNT.DTE = 1` (channel enable)
00026 * 3. Set `DMREQ.SWREQ = 1` (trigger transfer)
00027 *
00028 * ## Thread Safety
00029 *
00030 * rx_dmaca_transfer_poll() is NOT thread-safe. The caller must ensure that
00031 * only one transfer is active per channel at a time.
00032 *
00033 * ## NASA Power of 10
00034 *
00035 * | Rule | Status | Notes |
00036 * |-----|-----|-----|
00037 * | 1. Simple flow | PASS | No goto/recursion |
00038 * | 2. Bounded loops | PASS | Timeout loop bounded by timeout_cycles |
00039 * | 3. No dynamic mem | PASS | Zero malloc |
00040 * | 4. Short functions | PASS | All <= 60 lines |
00041 * | 5. Assertions | PASS | NULL checks + range checks |
00042 * | 6. Small scope | PASS | Vars declared at first use |
00043 * | 7. Check returns | PASS | All returns checked |
00044 * | 8. Limit preprocessor | PASS | C23 typed enums |
00045 * | 9. Restrict pointers | WARN | Function pointers via bus interface pattern |
00046 * | 10. Compiler warnings | PASS | -Wall -Wextra -Werror |
00047 *
00048 * ## Driver Lifecycle State Machine
00049 *
00050 * @startuml
00051 * [*] --> STOPPED : (after reset -- MSTPCRA bit 28 set)
00052 *
00053 * STOPPED --> ACTIVE : rx_dmaca_init() [k_rx_ok]\n(MSTPCRA bit 28=0, DMST=1)
00054 * STOPPED --> STOPPED : rx_dmaca_init() [double-init]\n/ k_rx_err_invalid_state
00055 * STOPPED --> STOPPED : rx_dmaca_deinit() [not init]\n/ k_rx_err_not_initialized
00056 *
00057 * ACTIVE --> TRANSFERRING : rx_dmaca_transfer_poll() [valid config]\n(DTE=1, SWREQ=1, poll ACT)
00058 * TRANSFERRING --> ACTIVE : ACT=0 (complete)\n(DTE cleared) / k_rx_ok
00059 * TRANSFERRING --> ACTIVE : timeout\n(DTE cleared) / k_rx_err_timeout
00060 *
00061 * ACTIVE --> ABORTING : rx_dmaca_abort(channel)
00062 * ABORTING --> ACTIVE : ACT=0 / k_rx_ok
00063 * ABORTING --> ACTIVE : timeout / k_rx_err_timeout
00064 *
00065 * ACTIVE --> STOPPED : rx_dmaca_deinit() [k_rx_ok]\n(DMST=0, MSTPCRA bit 28=1)
00066 * @enduml
00067 *
00068 * ## Transfer Sequence for rx_dmaca_transfer_poll()
00069 *
00070 * @startuml
00071 * participant "caller" as C
00072 * participant "rx_dmaca_transfer_poll" as T
00073 * participant "internal_configure_channel" as CF
00074 * participant "DMAC regs" as R
00075 * participant "internal_poll_act" as P
00076 * C -> T: rx_dmaca_transfer_poll(&config)
00077 * T -> T: validate: config, initialized,\nchannel, len, timeout, dst_addr, src
00078 * T -> CF: internal_configure_channel(&config)
00079 * CF -> R: DMSAR = src
00080 * CF -> R: DMDAR = dst_addr (e.g. CRCDIR)
00081 * CF -> R: DMCRA = len
00082 * CF -> R: DMTMD = normal|software|8-bit
00083 * CF -> R: DMAMD = src-increment|dst-fixed
00084 * CF -> R: DMINT = 0 (polling, no interrupts)
00085 * T -> R: DMCNT.DTE = 1 (enable channel)
00086 * T -> R: DMREQ = SWREQ|CLRS (trigger + auto-clear)
00087 * T -> P: internal_poll_act(channel, timeout_cycles)
00088 * loop check DMSTS.ACT (bounded by timeout_cycles)
00089 *   P -> R: read DMSTS
00090 *   alt ACT == 0
00091 *     P -> R: DMSTS = 0 (clear DTIF)
00092 *     P --> T: k_rx_ok
00093 *   else ACT == 1
00094 *     note over P: continue loop
00095 *   end
00096 * end
00097 * T -> R: DMCNT.DTE = 0 (always cleared on exit)
00098 * T -> C: k_rx_ok or k_rx_err_timeout
00099 * @enduml
00100 *
00101 * @see rx_crc.h CRC library that uses this driver for hw_dma backend
00102 * @see rx72n_dmac_regs.h Hardware register definitions
00103 *

```

```

00104 * @author Locked, Inc.
00105 * @date 2026-03-05
00106 * @copyright Copyright (c) 2026 Locked Inc.
00107 * SPDX-License-Identifier: MIT
00108 */
00109
00110 #pragma once
00111
00112 #include <stdint.h>
00113
00114 #include "rx_err.h"
00115
00116 #ifdef __cplusplus
00117 extern "C" {
00118 #endif
00119
00120 /*
=====
00121 * Constants
00122 *
=====
00123 */
00124
00125 /**
00126 * @enum rx_dmaca_defaults_t
00127 * @brief Default constants for the DMAC-CRC integration
00128 *
00129 * @details
00130 * Provides compile-time constants for CRC-specific DMA usage.
00131 *
00132 * @since Version 1.0.0
00133 */
00134 typedef enum : uint32_t {
00135 /**
00136 * @brief Timeout cycles for DMA polling (208 us at 240 MHz, 10x margin)
00137 * @details
00138 * Based on worst-case DMA transfer of 65535 bytes at 8-bit mode.
00139 * At ~240 MHz with ~6 cycles/byte, 65535 bytes ~ 1.6 ms = 384000 cycles.
00140 * 50000 cycles gives approximately 208 us which covers typical CRC buffers.
00141 * @par Value: 50000
00142 */
00143 k_dma_crc_timeout_cycles = 50000U,
00144
00145 /**
00146 * @brief DMAC channel reserved for CRC transfers
00147 * @details Channel 0 has highest priority among the 8 DMAC channels.
00148 * @par Value: 0
00149 */
00150 k_dma_channel_crc = 0U,
00151 } rx_dmaca_defaults_t;
00152
00153 /*
=====
00154 * Types
00155 *
=====
00156 */
00157
00158 /**
00159 * @struct rx_dmaca_config_t
00160 * @brief Configuration for a single DMAC polling transfer
00161 *
00162 * @details
00163 * Describes one DMA transfer from a source buffer to a fixed destination
00164 * address (e.g., CRCDIR at 0x00088284). All fields must be set before
00165 * calling rx_dmaca_transfer_poll().
00166 *
00167 * @invariant channel must be in [0, 7]
00168 * @invariant src must not be NULL
00169 * @invariant len must be in [1, 65535]
00170 * @invariant timeout_cycles must be > 0
00171 *
00172 * @code
00173 rx_dmaca_config_t cfg = {
00174     .channel    = k_dma_channel_crc,
00175     .src        = data,
00176     .len        = len,
00177     .dst_addr   = k_crcdir_addr,
00178     .timeout_cycles = k_dma_crc_timeout_cycles,
00179 };
00180 rx_err_t err = rx_dmaca_transfer_poll(&cfg);
00181 * @endcode
00182 *
00183 * @see rx_dmaca_transfer_poll() Uses this config
00184 *
00185 * @since Version 1.0.0
00186 */

```

```

00187 typedef struct {
00188     uint8_t    channel;          /**< DMA channel (0-7); 0 = highest priority */
00189     const uint8_t* src;          /**< Source buffer pointer (incremented by DMA) */
00190     uint32_t    len;             /**< Number of bytes to transfer [1, 65535] */
00191     uintptr_t    dst_addr;       /**< Destination address (fixed, e.g., CRCDIR) */
00192     uint32_t    timeout_cycles;  /**< Max poll iterations before timeout error */
00193 } rx_dmaca_config_t;
00194
00195 /*
=====
00196  * Public API
00197  *
=====
00198  */
00199
00200 /**
00201  * @brief Initialize the DMAC peripheral (enable module clock, set DMST=1)
00202  *
00203  * @details
00204  * Enables the DMAC module by clearing MSTPCRA bit 28 (module stop) then
00205  * setting DMAST.DMST = 1 to allow channel operation.
00206  *
00207  * Re-initialization returns k_rx_err_invalid_state (not k_rx_ok) to alert
00208  * callers of unexpected double-init. Callers that need idempotent behavior
00209  * should check the return value and treat k_rx_err_invalid_state as acceptable.
00210  *
00211  * @return rx_err_t
00212  * @retval k_rx_ok Peripheral initialized and ready
00213  * @retval k_rx_err_invalid_state Already initialized
00214  *
00215  * @pre System clock (PCLKA) must be running
00216  * @pre Protection registers must be accessible
00217  * @post MSTPCRA bit 28 cleared (DMAC clock enabled)
00218  * @post DMAST.DMST = 1 (module operating)
00219  *
00220  * @note Not thread-safe; call during single-threaded system init
00221  * @warning Do not call from interrupt context
00222  *
00223  * @since Version 1.0.0
00224  */
00225 [[nodiscard]] rx_err_t rx_dmaca_init(void);
00226
00227 /**
00228  * @brief Transfer data from src to a fixed destination via DMA (polling)
00229  *
00230  * @details
00231  * Configures DMAC channel, triggers a software-initiated transfer, then
00232  * polls DMSTS.ACT until transfer completes or timeout expires. Clears
00233  * DTE on completion or error.
00234  *
00235  * Transfer size (DMTMD.SZ) is always 8-bit (k_dmtmd_sz_8bit) for CRCDIR
00236  * compatibility. The CRCA peripheral processes 8-bit bytes regardless of
00237  * the 32-bit CRC polynomial selected.
00238  *
00239  * @param[in] config Transfer configuration (channel, src, len, dst, timeout)
00240  *
00241  * @return rx_err_t
00242  * @retval k_rx_ok Transfer complete
00243  * @retval k_rx_err_null_ptr config is NULL
00244  * @retval k_rx_err_invalid_arg channel >= 8, len == 0, len > 65535,
00245  *         timeout_cycles == 0 or > k_dmaca_timeout_cycles_max,
00246  *         or dst_addr == 0
00247  * @retval k_rx_err_not_initialized rx_dmaca_init() not called
00248  * @retval k_rx_err_timeout Transfer did not complete within timeout_cycles
00249  *
00250  * @pre rx_dmaca_init() must have been called
00251  * @pre config->src must point to valid readable memory of config->len bytes
00252  * @post On success, all config->len bytes written to config->dst_addr
00253  * @post DMCNT.DTE cleared regardless of success or failure
00254  *
00255  * @note Not thread-safe on the same channel
00256  *
00257  * @since Version 1.0.0
00258  */
00259 [[nodiscard]] rx_err_t rx_dmaca_transfer_poll(const rx_dmaca_config_t* config);
00260
00261 /**
00262  * @brief Abort an in-progress DMA transfer on a channel
00263  *
00264  * @details
00265  * Clears DMCNT.DTE immediately, then polls until DMSTS.ACT = 0 to ensure
00266  * the transfer has stopped before returning.
00267  *
00268  * @param[in] channel Channel to abort (0-7)
00269  *
00270  * @return rx_err_t
00271  * @retval k_rx_ok Channel stopped

```

```

00272 * @retval k_rx_err_invalid_arg channel >= 8
00273 * @retval k_rx_err_not_initialized rx_dmaca_init() not called
00274 * @retval k_rx_err_timeout ACT bit did not clear within timeout
00275 *
00276 * @pre rx_dmaca_init() must have been called
00277 * @pre channel must be in [0, 7]
00278 * @post DMCNT.DTE = 0 for the specified channel
00279 * @post DMSTS.ACT = 0 (transfer stopped)
00280 *
00281 * @since Version 1.0.0
00282 */
00283 [[nodiscard]] rx_err_t rx_dmaca_abort(uint8_t channel);
00284
00285 /**
00286 * @brief Deinitialize DMAC (clear DMST, lock MSTPCRA)
00287 *
00288 * @details
00289 * Sets DMAST.DMST = 0 then re-enables module stop (MSTPCRA bit 28 = 1)
00290 * to gate the DMAC clock for power saving.
00291 *
00292 * @return rx_err_t
00293 * @retval k_rx_ok Peripheral deinitialized
00294 * @retval k_rx_err_not_initialized Not initialized
00295 *
00296 * @pre rx_dmaca_init() must have been called
00297 * @pre No transfers are in progress on any channel
00298 * @post DMAST.DMST = 0 (all channels suspended)
00299 * @post MSTPCRA bit 28 = 1 (module stopped)
00300 *
00301 * @warning Calling deinit while a DMA transfer is in progress results in undefined
00302 *         behavior and potential data corruption at the destination address.
00303 *
00304 * @since Version 1.0.0
00305 */
00306 [[nodiscard]] rx_err_t rx_dmaca_deinit(void);
00307
00308 #ifdef __cplusplus
00309 }
00310 #endif

```

4.3 libs/rx_dmaca/src/rx_dmaca.c File Reference

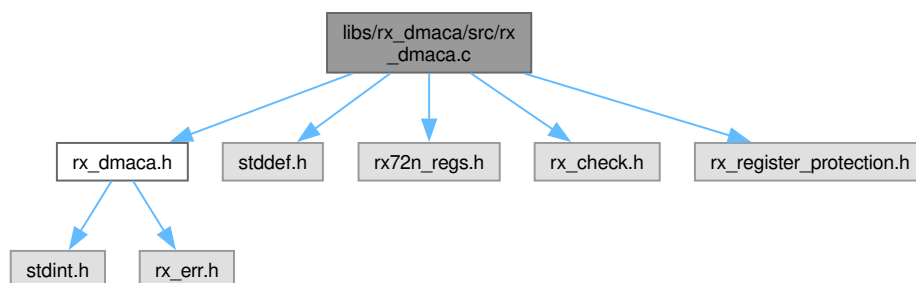
RX72N DMAC Polling-Mode Driver Implementation.

```

#include "rx_dmaca.h"
#include <stddef.h>
#include "rx72n_regs.h"
#include "rx_check.h"
#include "rx_register_protection.h"

```

Include dependency graph for rx_dmaca.c:



Enumerations

- enum `rx_dmaca_internal_constants_t` : `uint32_t` { `k_dmaca_abort_timeout_cycles` = 10000U , `k_dmaca_len_max` = 65535U , `k_dmaca_timeout_cycles_max` = 10000000U }

Internal constants for DMAC driver.

Functions

- static void [internal_configure_channel](#) (const [rx_dmaca_config_t](#) *config)
Configure DMAC channel registers for a transfer.
- static [rx_err_t](#) [internal_poll_act](#) ([uint8_t](#) channel, [uint32_t](#) timeout_cycles)
Poll DMSTS.ACT until clear or timeout.
- [rx_err_t](#) [rx_dmaca_init](#) (void)
Initialize the RX72N DMAC peripheral.
- [rx_err_t](#) [rx_dmaca_transfer_poll](#) (const [rx_dmaca_config_t](#) *config)
Perform a polled DMA transfer and block until complete or timeout.
- [rx_err_t](#) [rx_dmaca_abort](#) ([uint8_t](#) channel)
Abort an in-progress DMA transfer on the specified channel.
- [rx_err_t](#) [rx_dmaca_deinit](#) (void)
Deinitialize the RX72N DMAC peripheral.

Variables

- static const char *const [s_tag](#) = "DMAC"
Log tag for this module.
- static bool [s_dmaca_initialized](#) = false
Module initialization flag.

4.3.1 Detailed Description

RX72N DMAC Polling-Mode Driver Implementation.

Implements polling-mode DMA transfers for the RX72N DMAC peripheral. Used by [rx_crc](#) to offload byte-stream writes to CRCDIR.

Hardware access uses the register accessors from [rx72n_dmac_regs.h](#). In test builds (UNIT_TEST), these accessors return pointers to mock state ([tests/mocks/rx72n_dmac_regs.h](#) shadows the real header), allowing full logic testing without RX72N hardware.

See also

[rx_dmaca.h](#) Public API

[rx72n_dmac_regs.h](#) Register definitions

Author

Locked, Inc.

Date

2026-03-05

Copyright

Copyright (c) 2026 Locked Inc. SPDX-License-Identifier: MIT

Definition in file [rx_dmaca.c](#).

4.3.2 Enumeration Type Documentation

4.3.2.1 rx_dmaca_internal_constants_t

enum [rx_dmaca_internal_constants_t](#) : uint32_t

Internal constants for DMAC driver.

Abort timeout and DMA length constraints.

Since

Version 1.0.0

Enumerator

k_dmaca_abort_timeout_cycles	Abort poll iterations before timeout (fixed internal limit).
k_dmaca_len_max	Maximum DMA transfer length (DMCRA is 16-bit).
k_dmaca_timeout_cycles_max	Maximum caller-supplied timeout_cycles for internal_poll_act() . Provides a statically-provable upper bound on the internal_poll_act() loop so that NASA Rule 2 (bounded loops) is satisfied even though the loop bound is a runtime parameter. Callers are validated against this limit in rx_dmaca_transfer_poll() before the loop executes.

Definition at line 48 of file [rx_dmaca.c](#).

```

00048         : uint32_t {
00049     /** @brief Abort poll iterations before timeout (fixed internal limit) */
00050     k_dmaca_abort_timeout_cycles = 10000U,
00051
00052     /** @brief Maximum DMA transfer length (DMCRA is 16-bit) */
00053     k_dmaca_len_max = 65535U,
00054
00055     /**
00056      * @brief Maximum caller-supplied timeout_cycles for internal_poll_act()
00057      *
00058      * @details
00059      * Provides a statically-provable upper bound on the internal_poll_act() loop
00060      * so that NASA Rule 2 (bounded loops) is satisfied even though the loop bound
00061      * is a runtime parameter. Callers are validated against this limit in
00062      * rx_dmaca_transfer_poll() before the loop executes.
00063      */
00064     k_dmaca_timeout_cycles_max = 10000000U,
00065
00066 } rx_dmaca_internal_constants_t;

```

4.3.3 Function Documentation

4.3.3.1 internal_configure_channel()

```

void internal_configure_channel (
    const rx\_dmaca\_config\_t * config) [static]

```

Configure DMAC channel registers for a transfer.

Sets DMSAR, DMDAR, DMCRA, DMTMD, DMAMD, and DMINT for one polling transfer. Transfer size is always 8-bit (DMTMD.SZ=00b) for CRCDIR compatibility.

Precondition

config is not NULL and all fields are valid
[rx_dmaca_init\(\)](#) has been called

Postcondition

Channel registers configured for transfer
 DMINT = 0 (no interrupts in polling mode)

Note

Not thread-safe on the same channel

Since

Version 1.0.0

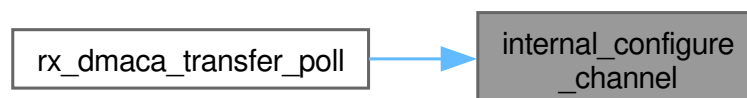
Definition at line 100 of file [rx_dmaca.c](#).

```
00101 {
00102     volatile rx_dmac_channel_regs_t* ch = dmac_channel(config->channel);
00103
00104     /* Source: increment after each read; Destination: fixed (CRCDIR) */
00105     ch->dmsar = (uint32_t)(uintptr_t)config->src;
00106     ch->dmdar = (uint32_t)config->dst_addr;
00107     ch->dmcra = config->len; /* Normal mode: lower 16 bits = transfer count */
00108     ch->dmtmd = (uint16_t)(k_dmtmd_md_normal | k_dmtmd_dctg_software | k_dmtmd_sz_8bit);
00109     ch->dmamd = (uint16_t)(k_dmamd_sm_increment | k_dmamd_dm_fixed);
00110     ch->dmint = k_dmint_disabled; /* No interrupts in polling mode */
00111 }
```

References [rx_dmaca_config_t::channel](#), [rx_dmaca_config_t::dst_addr](#), [rx_dmaca_config_t::len](#), and [rx_dmaca_config_t::src](#).

Referenced by [rx_dmaca_transfer_poll\(\)](#).

Here is the caller graph for this function:



4.3.3.2 internal_poll_act()

```
rx_err_t internal_poll_act (
    uint8_t channel,
    uint32_t timeout_cycles) [static]
```

Poll DMSTS.ACT until clear or timeout.

Loops up to timeout_cycles times checking if the DMA ACT flag (transfer in progress) is clear. Returns `k_rx_ok` on success, `k_rx_err_timeout` if the loop exhausts without ACT clearing.

Precondition

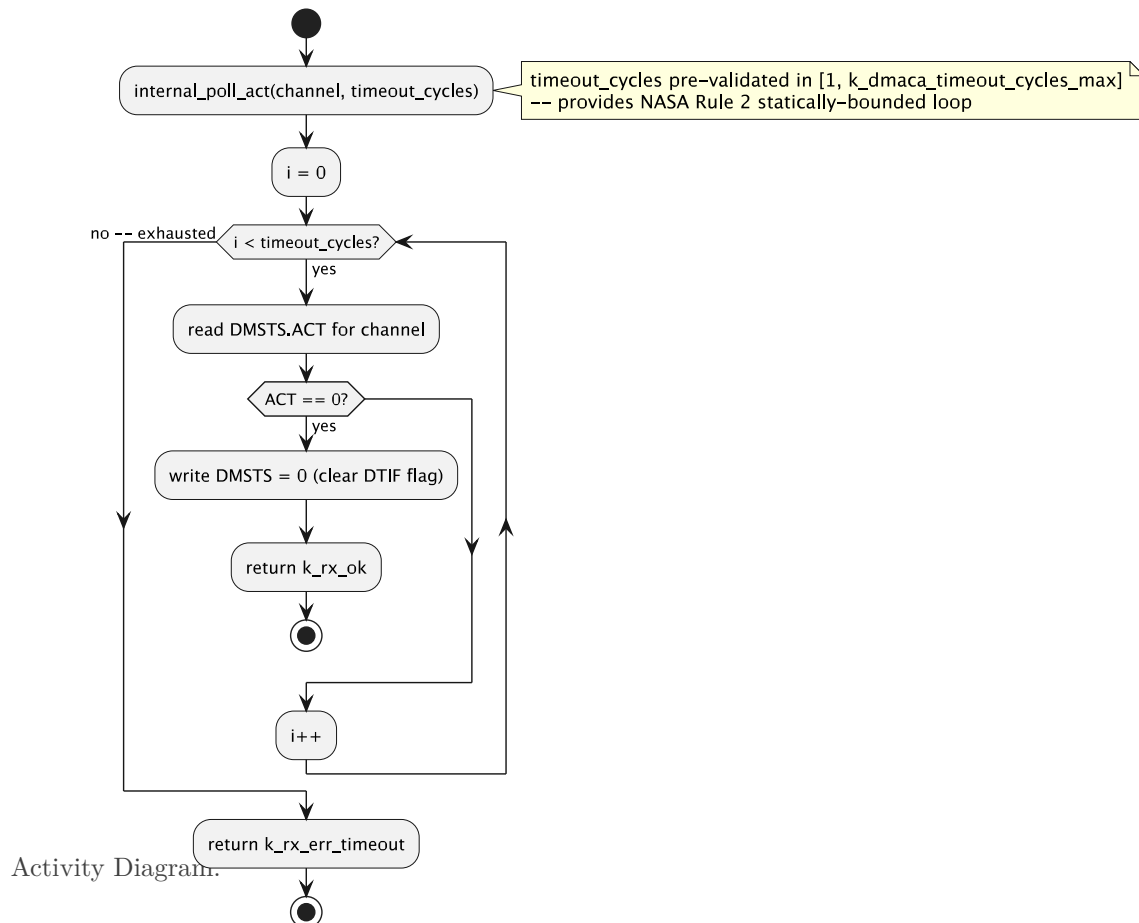
`channel < k_dmac_channel_count`

`timeout_cycles > 0` and `<= k_dmaca_timeout_cycles_max` (enforced by caller)

Postcondition

On `k_rx_ok`: `DMSTS.ACT = 0`

On `k_rx_err_timeout`: `DMSTS.ACT` may still be set; caller must clear DTE



Note

Not thread-safe on the same channel

Since

Version 1.0.0

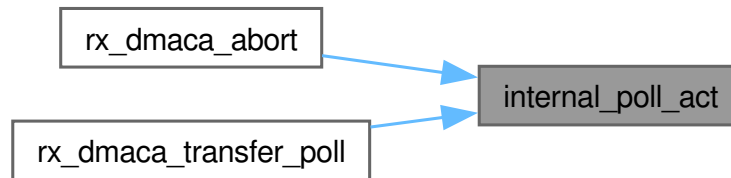
Definition at line 153 of file [rx_dmaca.c](#).

```

00154 {
00155     for (uint32_t i = 0U; i < timeout_cycles; i++) {
00156         if ((dmac_channel(channel)->dmsts & k_dmsts_act) == 0U) {
00157             dmac_channel(channel)->dmsts = 0U; /* Clear DTIF flag after ACT=0 */
00158             return k_rx_ok;
00159         }
00160     }
00161     return k_rx_err_timeout;
00162 }
  
```

Referenced by [rx_dmaca_abort\(\)](#), and [rx_dmaca_transfer_poll\(\)](#).

Here is the caller graph for this function:



4.3.3.3 rx`dmaca`abort()

```
rx_err_t rx_dmaca_abort (
    uint8_t channel) [nodiscard]
```

Abort an in-progress DMA transfer on the specified channel.

Abort an in-progress DMA transfer on a channel.

Immediately clears DTE to disable the channel, then polls the ACT bit until the active transfer drains or the internal abort timeout expires. Returns `k_rx_ok` when the channel is confirmed idle.

Precondition

`rx_dmaca_init()` must have been called successfully
 channel must be in `[0, k_dmac_channel_count - 1]`

Postcondition

DTE cleared for the specified channel
 Channel ACT bit is 0 on `k_rx_ok` return

Note

Not thread-safe; do not call concurrently on the same channel

Since

Version 1.0.0

Definition at line 330 of file `rx_dmaca.c`.

```
00331 {
00332     /* Pre-conditions: validate inputs (NASA Rule 5) */
00333     if (!s_dmaca_initialized) {
00334         rx_log_error(s_tag, "DMAC not initialized");
00335         return k_rx_err_not_initialized;
00336     }
00337
00338     if (channel >= k_dmac_channel_count) {
00339         return k_rx_err_invalid_arg;
00340     }
00341
00342     /* Clear DTE to disable channel immediately */
00343     dmac_channel(channel)->dmcnt = 0U;
00344
00345     /* Wait for active transfer to drain (polls ACT bit) */
00346     return internal_poll_act(channel, k_dmaca_abort_timeout_cycles);
00347 }
```

References `internal_poll_act()`, `k_dmaca_abort_timeout_cycles`, `s_dmaca_initialized`, and `s_tag`.

Here is the call graph for this function:



4.3.3.4 rx_dmaca_deinit()

```
rx_err_t rx_dmaca_deinit (
    void ) [nodiscard]
```

Deinitialize the RX72N DMAC peripheral.

Deinitialize DMAC (clear DMST, lock MSTPCRA).

Clears DMAST.DMST (suspends all channels), unlocks PRCR, sets MSTPCRA bit 28 to gate the DMAC module clock, re-locks PRCR, and marks the driver as uninitialized. Returns `k_rx_err_not_initialized` if called without a prior [rx_dmaca_init\(\)](#).

Precondition

[rx_dmaca_init\(\)](#) must have been called successfully
No DMA transfers should be in progress (all channels idle)

Postcondition

`s_dmaca_initialized` set to false
DMAC module clock disabled (MSTPCRA bit 28 set)

Note

Not thread-safe; call during single-threaded system teardown

Since

Version 1.0.0

Definition at line 367 of file [rx_dmaca.c](#).

```
00368 {
00369     /* Pre-condition: must be initialized (NASA Rule 5) */
00370     if (!s_dmaca_initialized) {
00371         rx_log_error(s_tag, "DMAC not initialized");
00372         return k_rx_err_not_initialized;
00373     }
00374
00375     /* Clear DMST to suspend all channels */
00376     *dmac_dmast_reg() = 0U;
00377
00378     /* Re-enable module stop to gate DMAC clock (power saving) */
00379     *prcr_reg() = k_rx_prcr_unlock_prc1;
00380     system_regs()->mstpcra |= k_dmac_mstpcra_bit;
00381     *prcr_reg() = k_rx_prcr_lock;
00382
00383     /* Post-condition: module stopped */
00384     s_dmaca_initialized = false;
00385     return k_rx_ok;
00386 }
```

References [s_dmaca_initialized](#), and [s_tag](#).

4.3.3.5 rx`dmaca`init()

```
rx_err_t rx_dmaca_init (
    void ) [nodiscard]
```

Initialize the RX72N DMAC peripheral.

Initialize the DMAC peripheral (enable module clock, set DMST=1).

Unlocks PRCR, enables the DMAC module clock (clears MSTPCRA bit 28), starts the DMAC module (sets DMAST.DMST = 1), and marks the driver as initialized. Returns k_rx_err_invalid_state on double-init without touching hardware.

Precondition

DMAC must not already be initialized (s_dmaca_initialized == false)
System clock (PCLKA) must be configured and running

Postcondition

s_dmaca_initialized set to true
DMAC module clock enabled and DMAST.DMST = 1

Note

Not thread-safe; call during single-threaded system initialization

Since

Version 1.0.0

Definition at line 186 of file [rx_dmaca.c](#).

```
00187 {
00188     /* Pre-condition: detect double-init (NASA Rule 5) */
00189     if (s_dmaca_initialized) {
00190         return k_rx_err_invalid_state;
00191     }
00192
00193     /* Enable DMAC module clock (clear MSTPCRA bit 28) */
00194     *prcr_reg() = k_rx_prcr_unlock_prc1;
00195     system_regs()->mstpcra &= ~k_dmac_mstpcra_bit;
00196     *prcr_reg() = k_rx_prcr_lock;
00197
00198     /* Start DMAC module (DMAST.DMST = 1) */
00199     *dmac_dmast_reg() = k_dmast_dmst;
00200
00201     /* Post-condition: module operational */
00202     s_dmaca_initialized = true;
00203     return k_rx_ok;
00204 }
```

References [s_dmaca_initialized](#).

4.3.3.6 rx_dmaca_transfer_poll()

```
rx_err_t rx_dmaca_transfer_poll (
    const rx_dmaca_config_t * config)  [nodiscard]
```

Perform a polled DMA transfer and block until complete or timeout.

Transfer data from src to a fixed destination via DMA (polling).

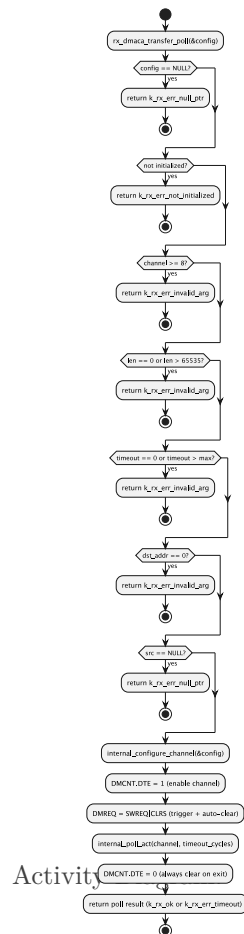
Validates all inputs, configures the DMA channel registers via [internal_configure_channel\(\)](#), starts the transfer (DTE=1, SWREQ=1), and polls the ACT bit until completion or timeout. Clears DTE on exit regardless of outcome.

Precondition

[rx_dmaca_init\(\)](#) must have been called successfully
 config->src must point to at least config->len bytes of readable memory

Postcondition

DMA channel DTE cleared (channel idle) regardless of return value
 On k_rx_ok, destination memory (config->dst_addr) contains transferred data



Note

Not thread-safe; concurrent calls on the same channel corrupt the transfer

Since

Version 1.0.0

Definition at line 266 of file [rx_dmaca.c](#).

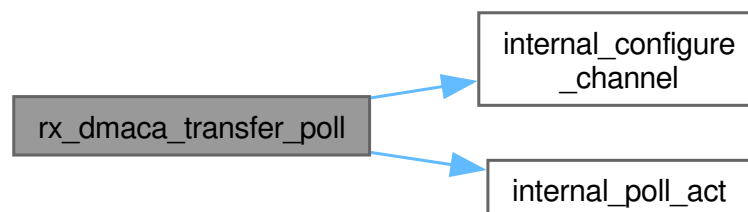
```

00267 {
00268     /* Pre-conditions: validate all inputs (NASA Rule 5) */
00269     RX_CHECK_NULL_PTR(config, s_tag, "config pointer is NULL");
00270
00271     if (!s_dmaca_initialized) {
00272         return k_rx_err_not_initialized;
00273     }
00274
00275     if (config->channel >= k_dmac_channel_count) {
00276         return k_rx_err_invalid_arg;
00277     }
00278
00279     if (config->len == 0U || config->len > k_dmaca_len_max) {
00280         return k_rx_err_invalid_arg;
00281     }
00282
00283     if (config->timeout_cycles == 0U || config->timeout_cycles > k_dmaca_timeout_cycles_max) {
00284         return k_rx_err_invalid_arg;
00285     }
00286
00287     if (config->dst_addr == 0U) {
00288         return k_rx_err_invalid_arg;
00289     }
00290
00291     RX_CHECK_NULL_PTR(config->src, s_tag, "src pointer is NULL");
00292
00293     /* Configure channel registers */
00294     internal_configure_channel(config);
00295
00296     /* DMA start sequence (mandatory order per HW manual section 18): */
00297     /* 1. DTE = 1 (channel enable) */
00298     dmac_channel(config->channel)->dmcnt = k_dmcnt_dte;
00299
00300     /* 2. SWREQ = 1 + CLRS = 1 (trigger and auto-clear) */
00301     dmac_channel(config->channel)->dmreq = (uint8_t)(k_dmreq_swreq | k_dmreq_clrs);
00302
00303     /* Poll until complete or timeout */
00304     rx_err_t ret = internal_poll_act(config->channel, config->timeout_cycles);
00305
00306     /* Post-condition: clear DTE regardless of outcome (NASA Rule 5) */
00307     dmac_channel(config->channel)->dmcnt = 0U;
00308
00309     return ret;
00310 }

```

References [rx_dmaca_config_t::channel](#), [rx_dmaca_config_t::dst_addr](#), [internal_configure_channel\(\)](#), [internal_poll_act\(\)](#), [k_dmaca_len_max](#), [k_dmaca_timeout_cycles_max](#), [rx_dmaca_config_t::len](#), [s_dmaca_initialized](#), [s_tag](#), [rx_dmaca_config_t::src](#), and [rx_dmaca_config_t::timeout_cycles](#).

Here is the call graph for this function:



4.3.4 Variable Documentation

4.3.4.1 s`dmaca`initialized

```
bool s_dmaca_initialized = false [static]
```

Module initialization flag.

Definition at line 77 of file [rx_dmaca.c](#).

Referenced by [rx_dmaca_abort\(\)](#), [rx_dmaca_deinit\(\)](#), [rx_dmaca_init\(\)](#), and [rx_dmaca_transfer_poll\(\)](#).

4.3.4.2 s`tag

```
const char* const s_tag = "DMAC" [static]
```

Log tag for this module.

Definition at line 74 of file [rx_dmaca.c](#).

Referenced by [rx_dmaca_abort\(\)](#), [rx_dmaca_deinit\(\)](#), and [rx_dmaca_transfer_poll\(\)](#).

4.4 rx`dmaca.c

[Go to the documentation of this file.](#)

```
00001 /**
00002  * @file rx_dmaca.c
00003  * @brief RX72N DMAC Polling-Mode Driver Implementation
00004  *
00005  * @details
00006  * Implements polling-mode DMA transfers for the RX72N DMAC peripheral.
00007  * Used by rx_crc to offload byte-stream writes to CRCDIR.
00008  *
00009  * Hardware access uses the register accessors from rx72n_dmac_regs.h.
00010  * In test builds (UNIT_TEST), these accessors return pointers to mock state
00011  * (tests/mocks/rx72n_dmac_regs.h shadows the real header), allowing full
00012  * logic testing without RX72N hardware.
00013  *
00014  * @see rx_dmaca.h Public API
00015  * @see rx72n_dmac_regs.h Register definitions
00016  *
00017  * @author Locked, Inc.
00018  * @date 2026-03-05
00019  * @copyright Copyright (c) 2026 Locked Inc.
00020  * SPDX-License-Identifier: MIT
00021  */
00022
00023 #include "rx_dmaca.h"
00024
00025 #include <stddef.h>
00026
00027 #ifdef UNIT_TEST
00028 #include "mock_rx72n_regs.h"
00029 #else
00030 #include "rx72n_regs.h"
00031 #endif
00032 #include "rx_check.h"
00033 #include "rx_register_protection.h"
00034
00035 /**
=====
00036  * Private Constants
00037  *
=====
00038  */
00039
00040 /**
```

```

00041 * @enum rx_dmaca_internal_constants_t
00042 * @brief Internal constants for DMAC driver
00043 *
00044 * @details Abort timeout and DMA length constraints.
00045 *
00046 * @since Version 1.0.0
00047 */
00048 typedef enum : uint32_t {
00049     /** @brief Abort poll iterations before timeout (fixed internal limit) */
00050     k_dmaca_abort_timeout_cycles = 10000U,
00051
00052     /** @brief Maximum DMA transfer length (DMCRA is 16-bit) */
00053     k_dmaca_len_max = 65535U,
00054
00055     /**
00056      * @brief Maximum caller-supplied timeout_cycles for internal_poll_act()
00057      *
00058      * @details
00059      * Provides a statically-provable upper bound on the internal_poll_act() loop
00060      * so that NASA Rule 2 (bounded loops) is satisfied even though the loop bound
00061      * is a runtime parameter. Callers are validated against this limit in
00062      * rx_dmaca_transfer_poll() before the loop executes.
00063      */
00064     k_dmaca_timeout_cycles_max = 10000000U,
00065 } rx_dmaca_internal_constants_t;
00066
00067
00068 /*
00069  * Module State
00070  */
00071
00072
00073 /** @brief Log tag for this module */
00074 static const char* const s_tag = "DMAC";
00075
00076 /** @brief Module initialization flag */
00077 static bool s_dmaca_initialized = false;
00078
00079
00080 * Internal Helpers
00081
00082
00083
00084 /**
00085  * @brief Configure DMAC channel registers for a transfer
00086  *
00087  * @details
00088  * Sets DMSAR, DMDAR, DMCRA, DMTMD, DMAMD, and DMINT for one polling transfer.
00089  * Transfer size is always 8-bit (DMTMD.SZ=00b) for CRCDIR compatibility.
00090  *
00091  * @pre config is not NULL and all fields are valid
00092  * @pre rx_dmaca_init() has been called
00093  * @post Channel registers configured for transfer
00094  * @post DMINT = 0 (no interrupts in polling mode)
00095  *
00096  * @note Not thread-safe on the same channel
00097  * @since Version 1.0.0
00098  */
00099
00100 static void internal_configure_channel(const rx_dmaca_config_t* config)
00101 {
00102     volatile rx_dmac_channel_regs_t* ch = dmac_channel(config->channel);
00103
00104     /** Source: increment after each read; Destination: fixed (CRCDIR) */
00105     ch->dmsar = (uint32_t)(uintptr_t)config->src;
00106     ch->dmdar = (uint32_t)config->dst_addr;
00107     ch->dmcra = config->len; /** Normal mode: lower 16 bits = transfer count */
00108     ch->dmtmd = (uint16_t)(k_dmtmd_md_normal | k_dmtmd_dctg_software | k_dmtmd_sz_8bit);
00109     ch->dmamd = (uint16_t)(k_dmamd_sm_increment | k_dmamd_dm_fixed);
00110     ch->dmint = k_dmint_disabled; /** No interrupts in polling mode */
00111 }
00112
00113 /**
00114  * @brief Poll DMSTS.ACT until clear or timeout
00115  *
00116  * @details
00117  * Loops up to timeout_cycles times checking if the DMA ACT flag
00118  * (transfer in progress) is clear. Returns k_rx_ok on success,
00119  * k_rx_err_timeout if the loop exhausts without ACT clearing.
00120  *
00121  *
00122  *
00123  * @pre channel < k_dmac_channel_count

```

```

00124 * @pre timeout_cycles > 0 and <= k_dmaca_timeout_cycles_max (enforced by caller)
00125 * @post On k_rx_ok: DMSTS.ACT = 0
00126 * @post On k_rx_err_timeout: DMSTS.ACT may still be set; caller must clear DTE
00127 *
00128 * @par Activity Diagram:
00129 * @startuml
00130 * start
00131 * :internal_poll_act(channel, timeout_cycles);
00132 * note right
00133 *   timeout_cycles pre-validated in [1, k_dmaca_timeout_cycles_max]
00134 *   -- provides NASA Rule 2 statically-bounded loop
00135 * end note
00136 * :i = 0;
00137 * while (i < timeout_cycles?) is (yes)
00138 *   :read DMSTS.ACT for channel;
00139 *   if (ACT == 0?) then (yes)
00140 *     :write DMSTS = 0 (clear DTIF flag);
00141 *     :return k_rx_ok;
00142 *   stop
00143 *   endif
00144 *   :i++;
00145 * endwhile (no -- exhausted)
00146 * :return k_rx_err_timeout;
00147 * stop
00148 * @enduml
00149 *
00150 * @note Not thread-safe on the same channel
00151 * @since Version 1.0.0
00152 */
00153 static rx_err_t internal_poll_act(uint8_t channel, uint32_t timeout_cycles)
00154 {
00155     for (uint32_t i = 0U; i < timeout_cycles; i++) {
00156         if ((dmac_channel(channel)->dmsts & k_dmsts_act) == 0U) {
00157             dmac_channel(channel)->dmsts = 0U; /* Clear DTIF flag after ACT=0 */
00158             return k_rx_ok;
00159         }
00160     }
00161     return k_rx_err_timeout;
00162 }
00163
00164 /*
=====
00165 * Public API
00166 *
=====
00167 */
00168 /**
00169 */
00170 * @brief Initialize the RX72N DMAC peripheral
00171 *
00172 * @details
00173 * Unlocks PRCR, enables the DMAC module clock (clears MSTPCRA bit 28), starts
00174 * the DMAC module (sets DMAST.DMST = 1), and marks the driver as initialized.
00175 * Returns k_rx_err_invalid_state on double-init without touching hardware.
00176 *
00177 *
00178 * @pre DMAC must not already be initialized (s_dmaca_initialized == false)
00179 * @pre System clock (PCLKA) must be configured and running
00180 * @post s_dmaca_initialized set to true
00181 * @post DMAC module clock enabled and DMAST.DMST = 1
00182 *
00183 * @note Not thread-safe; call during single-threaded system initialization
00184 * @since Version 1.0.0
00185 */
00186 rx_err_t rx_dmaca_init(void)
00187 {
00188     /* Pre-condition: detect double-init (NASA Rule 5) */
00189     if (s_dmaca_initialized) {
00190         return k_rx_err_invalid_state;
00191     }
00192
00193     /* Enable DMAC module clock (clear MSTPCRA bit 28) */
00194     *prcr_reg() = k_rx_prcr_unlock_prc1;
00195     system_regs()->mstpcra &= ~k_dmac_mstpcra_bit;
00196     *prcr_reg() = k_rx_prcr_lock;
00197
00198     /* Start DMAC module (DMAST.DMST = 1) */
00199     *dmac_dmast_reg() = k_dmast_dmst;
00200
00201     /* Post-condition: module operational */
00202     s_dmaca_initialized = true;
00203     return k_rx_ok;
00204 }
00205
00206 /**
00207 * @brief Perform a polled DMA transfer and block until complete or timeout
00208 *

```



```

00209 * @details
00210 * Validates all inputs, configures the DMA channel registers via
00211 * internal_configure_channel(), starts the transfer (DTE=1, SWREQ=1),
00212 * and polls the ACT bit until completion or timeout. Clears DTE on exit
00213 * regardless of outcome.
00214 *
00215 *
00216 *
00217 * @pre rx_dmaca_init() must have been called successfully
00218 * @pre config->src must point to at least config->len bytes of readable memory
00219 * @post DMA channel DTE cleared (channel idle) regardless of return value
00220 * @post On k_rx_ok, destination memory (config->dst_addr) contains transferred data
00221 *
00222 * @par Activity Diagram:
00223 * @startuml
00224 * start
00225 * :rx_dmaca_transfer_poll(&config);
00226 * if (config == NULL?) then (yes)
00227 *   :return k_rx_err_null_ptr;
00228 * stop
00229 * endif
00230 * if (not initialized?) then (yes)
00231 *   :return k_rx_err_not_initialized;
00232 * stop
00233 * endif
00234 * if (channel >= 8?) then (yes)
00235 *   :return k_rx_err_invalid_arg;
00236 * stop
00237 * endif
00238 * if (len == 0 or len > 65535?) then (yes)
00239 *   :return k_rx_err_invalid_arg;
00240 * stop
00241 * endif
00242 * if (timeout == 0 or timeout > max?) then (yes)
00243 *   :return k_rx_err_invalid_arg;
00244 * stop
00245 * endif
00246 * if (dst_addr == 0?) then (yes)
00247 *   :return k_rx_err_invalid_arg;
00248 * stop
00249 * endif
00250 * if (src == NULL?) then (yes)
00251 *   :return k_rx_err_null_ptr;
00252 * stop
00253 * endif
00254 * :internal_configure_channel(&config);
00255 * :DMCNT.DTE = 1 (enable channel);
00256 * :DMREQ = SWREQ|CLRS (trigger + auto-clear);
00257 * :internal_poll_act(channel, timeout_cycles);
00258 * :DMCNT.DTE = 0 (always clear on exit);
00259 * :return poll result (k_rx_ok or k_rx_err_timeout);
00260 * stop
00261 * @enduml
00262 *
00263 * @note Not thread-safe; concurrent calls on the same channel corrupt the transfer
00264 * @since Version 1.0.0
00265 */
00266 rx_err_t rx_dmaca_transfer_poll(const rx_dmaca_config_t* config)
00267 {
00268     /* Pre-conditions: validate all inputs (NASA Rule 5) */
00269     RX_CHECK_NULL_PTR(config, s_tag, "config pointer is NULL");
00270
00271     if (!s_dmaca_initialized) {
00272         return k_rx_err_not_initialized;
00273     }
00274
00275     if (config->channel >= k_dmac_channel_count) {
00276         return k_rx_err_invalid_arg;
00277     }
00278
00279     if (config->len == 0U || config->len > k_dmaca_len_max) {
00280         return k_rx_err_invalid_arg;
00281     }
00282
00283     if (config->timeout_cycles == 0U || config->timeout_cycles > k_dmaca_timeout_cycles_max) {
00284         return k_rx_err_invalid_arg;
00285     }
00286
00287     if (config->dst_addr == 0U) {
00288         return k_rx_err_invalid_arg;
00289     }
00290
00291     RX_CHECK_NULL_PTR(config->src, s_tag, "src pointer is NULL");
00292
00293     /* Configure channel registers */
00294     internal_configure_channel(config);
00295

```

```

00296 /* DMA start sequence (mandatory order per HW manual section 18): */
00297 /* 1. DTE = 1 (channel enable) */
00298 dmact_channel(config->channel)->dmcnt = k_dmcnt_dte;
00299
00300 /* 2. SWREQ = 1 + CLRS = 1 (trigger and auto-clear) */
00301 dmact_channel(config->channel)->dmreq = (uint8_t)(k_dmreq_swreq | k_dmreq_clr);
00302
00303 /* Poll until complete or timeout */
00304 rx_err_t ret = internal_poll_act(config->channel, config->timeout_cycles);
00305
00306 /* Post-condition: clear DTE regardless of outcome (NASA Rule 5) */
00307 dmact_channel(config->channel)->dmcnt = 0U;
00308
00309 return ret;
00310 }
00311
00312 /**
00313  * @brief Abort an in-progress DMA transfer on the specified channel
00314  *
00315  * @details
00316  * Immediately clears DTE to disable the channel, then polls the ACT bit
00317  * until the active transfer drains or the internal abort timeout expires.
00318  * Returns k_rx_ok when the channel is confirmed idle.
00319  *
00320  *
00321  *
00322  * @pre rx_dmact_init() must have been called successfully
00323  * @pre channel must be in [0, k_dmact_channel_count - 1]
00324  * @post DTE cleared for the specified channel
00325  * @post Channel ACT bit is 0 on k_rx_ok return
00326  *
00327  * @note Not thread-safe; do not call concurrently on the same channel
00328  * @since Version 1.0.0
00329  */
00330 rx_err_t rx_dmact_abort(uint8_t channel)
00331 {
00332     /* Pre-conditions: validate inputs (NASA Rule 5) */
00333     if (!s_dmact_initialized) {
00334         rx_log_error(s_tag, "DMAC not initialized");
00335         return k_rx_err_not_initialized;
00336     }
00337
00338     if (channel >= k_dmact_channel_count) {
00339         return k_rx_err_invalid_arg;
00340     }
00341
00342     /* Clear DTE to disable channel immediately */
00343     dmact_channel(channel)->dmcnt = 0U;
00344
00345     /* Wait for active transfer to drain (polls ACT bit) */
00346     return internal_poll_act(channel, k_dmact_abort_timeout_cycles);
00347 }
00348
00349 /**
00350  * @brief Deinitialize the RX72N DMAC peripheral
00351  *
00352  * @details
00353  * Clears DMAST.DMST (suspends all channels), unlocks PRCR, sets MSTPCRA
00354  * bit 28 to gate the DMAC module clock, re-locks PRCR, and marks the
00355  * driver as uninitialized. Returns k_rx_err_not_initialized if called
00356  * without a prior rx_dmact_init().
00357  *
00358  *
00359  * @pre rx_dmact_init() must have been called successfully
00360  * @pre No DMA transfers should be in progress (all channels idle)
00361  * @post s_dmact_initialized set to false
00362  * @post DMAC module clock disabled (MSTPCRA bit 28 set)
00363  *
00364  * @note Not thread-safe; call during single-threaded system teardown
00365  * @since Version 1.0.0
00366  */
00367 rx_err_t rx_dmact_deinit(void)
00368 {
00369     /* Pre-condition: must be initialized (NASA Rule 5) */
00370     if (!s_dmact_initialized) {
00371         rx_log_error(s_tag, "DMAC not initialized");
00372         return k_rx_err_not_initialized;
00373     }
00374
00375     /* Clear DMST to suspend all channels */
00376     *dmact_dmast_reg() = 0U;
00377
00378     /* Re-enable module stop to gate DMAC clock (power saving) */
00379     *prcr_reg() = k_rx_prcr_unlock_prc1;
00380     system_regs()->mstpcra |= k_dmact_mstpcra_bit;
00381     *prcr_reg() = k_rx_prcr_lock;
00382 }

```

```
00383  /* Post-condition: module stopped */
00384  s_dmaca_initialized = false;
00385  return k_rx_ok;
00386 }
```

